

1. Product Overview

What problem this product solves

`campaign-pilot-ai-dashboard` solves the problem of turning raw retail customer and order data into a revenue-driving marketing campaign with minimal manual setup. Instead of asking the user to build segments, write campaign copy, or forecast ROI, it:

- ingests customer/order CSV data,
- discovers high-value segments,
- interprets a marketer's prompt,
- chooses the best audience,
- generates campaign content,
- simulates launch outcomes,
- and surfaces expected revenue and ROI.

Who the users are

- Retail growth marketers
- E-commerce operators
- Revenue ops and demand generation teams
- Founders or operators preparing investor-demo campaigns
- Anyone who wants quick marketing strategy from customer behavior rather than a full CRM workflow

Why it is AI-native

It is AI-native because the core product experience is built around AI reasoning and generation, not just automation:

- GoalInterpreterAgent.ts interprets the business prompt
- AudienceAgent.ts selects the best customer segment
- ContentAgent.ts generates email/SMS copy
- route.ts uses AI to explain why a segment was chosen
- OpenRouterProvider.ts plugs into OpenRouter/Gemini to power those decisions

The AI is the primary decision layer, not a secondary feature.

How it differs from a traditional CRM

Traditional CRMs focus on:

- contact records
- lead lifecycles
- sales pipeline management
- manual audience selection
- generic campaign templates

This product instead focuses on:

- revenue opportunity discovery
- AI-assisted campaign strategy
- automated audience selection
- campaign narrative & ROI simulation
- one-click launch simulation

It is closer to an “AI campaign copilot” than a contact database.

Core value proposition

- Turn customer data into a campaign strategy in minutes
- Use AI to choose the highest-impact segment and reason about it
- Generate ready-to-send messaging for email + SMS
- Forecast revenue and ROI automatically
- Simulate campaign execution and show performance without real delivery infrastructure

2. End-to-End User Flow

This is the exact application path from website entry to ROI display.

Step 1: User opens website

- UI file: page.tsx
- Component: `HeroOnboarding`
- No API call yet
- Backend: none
- Database: none
- AI: none
- Data returned: initial page shell

Step 2: User uploads data / connects source

- UI file: data-sources-modal.tsx
- Route called: `POST /api/data/ingest` → route.ts
- Backend service: `CustomerIntelligenceService.discoverSegments()`
- Database tables touched:
 - `Customer`
 - `Order`
- AI calls: none
- Data returned:
 - `customersImported`
 - `ordersImported`
 - `totalRevenue`
 - `averageOrderValue`
 - `segments`
 - errors, ingestion stats

Step 3: System discovers segments

- Logic file: CustomerIntelligenceService.ts

- Process:
 - load customers and orders from prisma
 - compute purchase stats
 - build 4 segment buckets
- Uses:
 - `Customer`
 - `Order`
- Output returns to frontend through ingest API
- Frontend component: `DataAnalysisSummary`, `AIOpportunities`

Step 4: User enters goal prompt

- UI file: ai-hero.tsx
- Event: form submission triggers `handleAnalyze` in page.tsx
- Route called: `POST /api/campaigns/preview` → route.ts
- Backend services:
 - `CustomerIntelligenceService.discoverSegments()`
 - `OpenRouterProvider.generate()`
 - `StrategyEngine.analyzePrompt()`
 - `StrategyEngine.computeExpectedRevenue()`
- Database tables touched:
 - none directly during preview
- AI calls executed:
 - OpenRouter/Gemini prompt to select best segment and explain decision
- Data returned:
 - `success`
 - `userPrompt`
 - `analysis`
 - `segment`

- `expectedRevenue`
- `goalReasoning`
- `businessImpact`
- `aiAssets` like `emailSubject`, `emailCopy`, `smsCopy`
- Frontend displays: `AIExplainabilityPanel`

Step 5: User generates full campaign strategy

- UI file: page.tsx
- Route called: `POST /api/campaigns/generate` → route.ts
- Backend services and classes:
 - `GoalInterpreterAgent`
 - `AudienceAgent`
 - `ContentAgent`
 - `OpenRouterProvider`
- prisma via prisma.ts
- Database tables touched:
 - `Campaign`
 - `CampaignMetrics`
- AI calls executed:
 - Goal interpretation
 - Audience selection
 - Content generation
- Data returned:
 - `campaignId`
 - `strategy` object with goal/audience/content
- Frontend components:
 - updates `currentStrategy`
 - displays `AIReasoningEngine`

Step 6: User launches campaign

- UI file: ai-reasoning-engine.tsx
- Method: `handleLaunch` in page.tsx
- Route called: `POST /api/campaigns/launch` → route.ts
- Backend services:
 - `SnapshotService.createSnapshot()`
 - `ChannelService.processCampaign()`
 - prisma update campaign status
- Database tables touched:
 - `Campaign`
 - `SegmentSnapshot`
 - `CampaignMetrics`
- AI calls:
 - none
- Data returned:
 - `status: 'QUEUED'`
 - `audienceSize`
- UI moves into simulation step

Step 7: Campaign send is started

- Route called: `POST /api/channel/send` → route.ts
- Backend service: `ChannelService.processCampaign()`
- Behavior:
 - starts async simulation using `ChannelService.simulateDelivery()`
 - does not await actual campaign completion
- Tables touched later via callbacks

Step 8: Delivery events are recorded

- Route called: `POST /api/callbacks/channel` → route.ts
- Backend services:
 - `prisma.deliveryLog.create`
 - `prisma.campaignMetrics.update`
 - `prisma.campaign.update`
- Database tables touched:
 - `DeliveryLog`
 - `CampaignMetrics`
 - `Campaign`
- Data returned:
 - success status
- This route is called repeatedly by `ChannelService` simulated phases:
 - `DELIVERED`
 - `OPENED`
 - `CLICKED`
 - `CONVERSION`
 - `COMPLETED`

Step 9: Frontend polls status

- Method: `pollStatus` in page.tsx
- Route called: `GET /api/campaigns/status?id=...` → route.ts
- Backend:
 - loads `Campaign` with `metrics`
- Frontend updates:
 - `campaignStatus`
 - `metrics`

- `currentStrategy`
- UI components:
 - `CampaignExecutionCenter`
 - `ExecutiveDashboard`
- When status reaches `COMPLETED`, UI transitions automatically to the final ROI view

Step 10: ROI displayed

- Frontend files:
 - campaign-execution-center.tsx
 - executive-dashboard.tsx
- Data used:
 - `metrics`
 - `strategy`
 - `audienceSize`
 - `expectedRevenue`
- Outcome:
 - baseline revenue
 - projected revenue lift
 - after campaign revenue (baseline + lift)
 - ROI
 - campaign performance visualization
- The ROI chart now compares total business revenue before campaign and after campaign, not just incremental campaign revenue

3. Architecture Deep Dive

app/

This is the Next.js frontend shell and page composition.

- page.tsx: main orchestration of app state, step flow, API calls, campaign lifecycle
- layout.tsx: global HTML and font wrapper

app/api/

Serverless API endpoints for the product's runtime logic.

route.ts

- ingests customer and order CSVs
- normalizes headers
- saves records to `Customer` and `Order`
- returns segment analytics

route.ts

- preview path
- uses `CustomerIntelligenceService` and `OpenRouterProvider`
- returns AI-driven audience explanation and expected revenue

route.ts

- builds full campaign strategy
- orchestrates agents
- persists `Campaign` and `CampaignMetrics`

route.ts

- creates `SegmentSnapshot`
- updates campaign status
- triggers channel send

route.ts

- returns campaign status and metrics

route.ts

- endpoint for campaign delivery simulation

route.ts

- receives simulated delivery events
- updates campaign metrics and status

components/

UI building blocks.

- ai-hero.tsx: prompt entry and suggested goals
- ai-explainability-panel.tsx: preview explanation and rationale
- ai-reasoning-engine.tsx: campaign reasoning + launch CTA
- campaign-execution-center.tsx: execution simulation dashboard
- executive-dashboard.tsx: final ROI story and charts
- data-analysis-summary.tsx: summary of imported data
- data-sources-modal.tsx: data ingestion modal
- ai-opportunities.tsx: segment opportunity cards
- workflow-progress.tsx: step progress bar
- hero-onboarding.tsx: landing experience

components contains the product UX; almost all domain logic is in lib.

lib/

Domain logic and AI integration.

lib/ai/

- LLMProvider.ts: provider interface
- OpenRouterProvider.ts: actual OpenRouter/Gemini client

lib/agents/

Agent-based reasoning pipeline:

- GoalInterpreterAgent.ts
- AudienceAgent.ts
- ContentAgent.ts

Each agent is responsible for one step in campaign strategy.

lib/services/

Business services:

- CustomerIntelligenceService.ts: segment discovery and customer analysis
- StrategyEngine.ts: prompt analysis, revenue math, content fallback, segment reasoning
- SnapshotService.ts: snapshot persistence
- ChannelService.ts: campaign delivery simulation

prisma.ts

- single Prisma client export
- loads `DATABASE\_URL`
- uses query logging in non-production

types.ts

- domain interfaces used in agents and components:
 - `CampaignGoal`
 - `CampaignAudience`
 - `CampaignContent`
 - `CampaignStrategy`
 - `CustomerSegment`

prisma/

- schema.prisma: database model definitions
- `migrations/`: migration metadata

Folder responsibility separation

- app: high-level page and API endpoints
- components: presentation and UX
- lib: business logic, AI orchestration, data access wrappers
- prisma: schema/database contract

This is a clean separation: UI just renders or calls APIs, backend routes orchestrate services, services implement the real domain behavior.

4. Database Deep Dive

Prisma models

`Customer`

Fields:

- `id`
- `email`
- `name`
- `orders`
- `snapshots`
- `createdAt`

Purpose:

- stores unique customer records
- primary entity for segmentation
- used during data ingestion and seg discovery

When updated:

- ingest route creates new customers
- order ingestion can create customers by email fallback

APIs:

- route.ts
- `CustomerIntelligenceService.discoverSegments()`
- route.ts fallback to all customers

`Order`

Fields:

- `id`
- `amount`
- `orderDate`
- `createdAt`
- `customerId`
- `customer`

Purpose:

- records purchase history
- provides spend, recency, weekend behavior analysis

When updated:

- ingest route adds orders
- required for segment discovery

APIs:

- route.ts
- `CustomerIntelligenceService.discoverSegments()`

- `ChannelService` average order value aggregation

`Campaign`

Fields:

- `id`
- `title`
- `status`
- `strategy`
- `snapshotId`
- `snapshot`
- `metrics`
- `deliveries`
- `createdAt`

Purpose:

- stores generated campaign strategy
- holds launch state
- connects to metrics and snapshot

When updated:

- created by `/api/campaigns/generate`
- status updated by `/api/campaigns/launch` and callbacks
- strategy JSON read by launch route and status route

APIs:

- `/api/campaigns/generate`
- `/api/campaigns/launch`
- `/api/campaigns/status`
- page.tsx polling

`SegmentSnapshot`

Fields:

- `id`
- `customerIds`
- `count`
- `createdAt`
- `campaigns`
- `customers`

Purpose:

- captures the exact audience for a launched campaign
- persists targeted customer IDs
- allows campaign state to reference the chosen snapshot

When updated:

- created by `/api/campaigns/launch`
- read by `SnapshotService.getCustomersInSnapshot()` if used

APIs:

- route.ts
- SnapshotService.ts

Note:

- `customerIds` is a JSON string, not normalized; this is an intentional performance shortcut.

`CampaignMetrics`

Fields:

- `id`
- `sent`

- `delivered`
- `opened`
- `clicked`
- `conversions`
- `revenue`
- `campaignId`
- `campaign`

Purpose:

- stores live campaign stats
- aggregates simulation events
- drives ROI dashboard UI

When updated:

- created on campaign generation
- updated by `/api/callbacks/channel`
- read by `/api/campaigns/status`

APIs:

- `/api/campaigns/generate`
- `/api/callbacks/channel`
- `/api/campaigns/status`

`DeliveryLog`

Fields:

- `id`
- `status`
- `channel`
- `campaignId`

- `campaign`
- `timestamp`

Purpose:

- event-level logging of delivery simulation
- useful for audit / debugging
- human-readable delivery timeline

When updated:

- inserted by `/api/callbacks/channel` for every event
- never read by current UI

APIs:

- `/api/callbacks/channel`

Relationships

- `Customer` → `Order`: one-to-many
- `Campaign` → `CampaignMetrics`: one-to-one
- `Campaign` → `SegmentSnapshot`: one-to-one
- `Campaign` → `DeliveryLog`: one-to-many

Data flow through tables

1. `Customer` and `Order` accumulate raw retail data.
2. `CustomerIntelligenceService` reads those tables to build segment summaries.
3. Preview/generate flows choose segment and compute revenue.
4. A launched campaign stores a snapshot of targeted customers.
5. Simulated delivery events feed into `CampaignMetrics` and `DeliveryLog`.
6. `ExecutiveDashboard` and polling query `CampaignMetrics` for results.

5. AI System Deep Dive

OpenRouter integration

- OpenRouterProvider.ts
- Uses `openai` Node SDK with `baseUrl: 'https://openrouter.ai/api/v1'`
- Model: `google/gemini-2.5-flash`
- Generates chat completions with:
 - `temperature: 0.7`
 - `max\_tokens: 800`

The provider is injected into runtime agents using the `LLMProvider` interface.

Which models are used

- `google/gemini-2.5-flash` via OpenRouter
- All AI text generation is routed through this model in:
 - route.ts
 - GoalInterpreterAgent.ts
 - AudienceAgent.ts
 - ContentAgent.ts

Every AI prompt and purpose

Preview Flow

File: route.ts

Prompt goal:

- interpret the user's prompt
- select the best segment
- explain why
- recommend action

- predict business impact

Response shape:

```
```json
{
 "intent":"CAMPAIGN",
 "objective":"",
 "selectedSegmentId":"",
 "segmentReasoning":"",
 "goalReasoning":"",
 "businessImpact":"",
 "recommendedAction":"",
 "emailSubject":"",
 "emailCopy":"",
 "smsCopy":""
}
```
```

Why this structure:

- enforces machine-readable output
- eases JSON extraction with regex
- keeps UI stable even if the text varies

Generate Campaign Flow

`GoalInterpreterAgent`

File: GoalInterpreterAgent.ts

Prompt goal:

- interpret the prompt in the context of actual revenue data

- choose CAMPAIGN vs ANALYSIS

- return objective, KPIs, reasoning

Response shape:

```
```json
{
 "intent": "CAMPAIGN",
 "objective": "",
 "kpis": ["KPI 1", "KPI 2"],
 "reasoning": ""
}
```
```

`AudienceAgent`

File: AudienceAgent.ts

Prompt goal:

- select the best segment for the user goal
- describe criteria and reasoning
- estimate business impact

Response shape:

```
```json
{
 "selectedSegmentId": "",
 "criteria": "",
 "reasoning": "",
 "businessImpact": ""
}
```

...

#### `ContentAgent`

File: ContentAgent.ts

Prompt goal:

- generate email subject, email copy, and SMS copy
- adapt content based on campaign intent and selected audience

Response shape:

```json

```
{  
  
  "subjectLine": "",  
  
  "messages": {  
  
    "email": "",  
  
    "sms": ""  
  
  },  
  
  "reasoning": ""  
  
}
```

...

Why prompts are structured this way

- The code uses `response.match(/\{[\s\S]\*\}/)` and `JSON.parse`
- therefore each prompt explicitly demands JSON only
- this reduces parsing brittleness
- it also makes the entire agent pipeline deterministic and traceable

Preview Flow

1. App sends prompt to `/api/campaigns/preview`
2. backend discovers segments

3. `OpenRouterProvider.generate()` receives a prompt listing segments
4. AI chooses `selectedSegmentId`
5. `computeExpectedRevenue()` projects revenue
6. response returns a preview object used by `AIExplainabilityPanel`

Generate Campaign Flow

1. `/api/campaigns/generate` creates `GoalInterpreterAgent`
2. `AudienceAgent` chooses segment and computes revenue
3. `ContentAgent` writes copy
4. campaign saved in `Campaign`
5. `CampaignMetrics` created with default zero values

Audience Selection Flow

- core logic is in `CustomerIntelligenceService.discoverSegments()`
- AI may select a segment based on prompt semantics and data summary
- if AI fails, fallback is the largest revenue segment
- selection is reinforced by `computeExpectedRevenue()` in `StrategyEngine.ts`

Content Generation Flow

- `ContentAgent` uses AI if available
- if AI fails, `StrategyEngine.getSegmentContent()` returns templates per segment:
 - `'weekend-shoppers'`
 - `'at-risk'`
 - `'high-value'`
 - `'recent-buyers'`
- fallback templates include subjectLine, email, SMS, offer code, discount
- AI is used to make content more unique and aligned to objective

ROI Flow

- Backend expected revenue is generated in preview and audience agent using `computeExpectedRevenue()`
- Frontend ROI is computed in `campaign-execution-center.tsx` and `executive-dashboard.tsx`
- the ROI story is a mix of:
 - predicted conversions
 - average order value
 - campaign cost
 - profit margin
- the product uses both AI output and deterministic formulas

6. Campaign Generation Walkthrough

User enters: `"Win back inactive customers"`

Step 1: Prompt submit

- UI: `ai-hero.tsx`
- method: `handleAnalyze(prompt)` in `page.tsx`
- calls `POST /api/campaigns/preview`

Step 2: Preview route logic

- File: `route.ts`
- `CustomerIntelligenceService.discoverSegments()` loads segments
- `analyzePrompt(prompt)` sets intent and objective
- AI prompt is built with:
 - retailer goal
 - available segment summary
- Gemini returns JSON containing `selectedSegmentId`
- expected revenue computed with:

- ``StrategyEngine.computeExpectedRevenue(segment, intelligence, segment.id)``
- likely segment selected for this prompt: ``at-risk``

Step 3: Preview display

- component: ``AIExplainabilityPanel``
- shows:
 - segment name
 - customer size
 - revenue contribution
 - expected revenue
- encourages user to generate campaign

Step 4: Generate campaign strategy

- ``handleGenerateStrategy`` in `page.tsx`
- calls ``POST /api/campaigns/generate``

Step 5: Agent pipeline

- ``GoalInterpreterAgent.execute()`` in `GoalInterpreterAgent.ts`
 - may generate objective like ``Win back inactive customers with a targeted re-engagement offer``
- ``AudienceAgent.execute()`` in `AudienceAgent.ts`
 - receives strategy and prompt
 - uses AI to choose ``selectedSegmentId``
 - likely returns segment:
 - ``at-risk``
 - criteria: customers inactive for 60+ days
 - expectedRevenue based on lift factor 0.25
- ``ContentAgent.execute()`` in `ContentAgent.ts`
 - generates email + SMS copy tailored to at-risk customers

Step 6: Persistence

- route.ts
- creates `Campaign`:
 - `title: Campaign: \${goalRes.output.objective}`
 - `status: 'DRAFT'`
 - `strategy: JSON.stringify(strategy)`
- creates `CampaignMetrics` with `campaignId`

Step 7: Launch

- ai-reasoning-engine.tsx click launch
- `handleLaunch()` in page.tsx
- calls `POST /api/campaigns/launch`

Step 8: Launch backend

- route.ts
- reads campaign strategy from DB
- extracts `customerIds` from `strategy.audience.output.customerIds`
- if absent, uses all customer IDs
- creates `SegmentSnapshot` with audience
- updates campaign status to `QUEUED`
- sets `CampaignMetrics.sent = audienceSize`
- calls `POST /api/channel/send`

Step 9: Simulated delivery

- route.ts
- `ChannelService.processCampaign()` begins `simulateDelivery`
- simulation phases:
 - `DELIVERED`

- `OPENED`
- `CLICKED`
- `CONVERSION`
- `COMPLETED`

Step 10: Callback updates

- route.ts
- each event:
 - logs `DeliveryLog`
 - increments metrics
 - sets campaign status to `EXECUTING`
- final phase sets `status = COMPLETED`

Step 11: UI updates

- page.tsx polls `/api/campaigns/status`
- `CampaignExecutionCenter` shows simulation progress
- when complete, `ExecutiveDashboard` displays ROI

7. Segment Discovery Logic

All segment discovery lives in CustomerIntelligenceService.ts.

High Value Customers

Calculated by:

- building spend list from every customer
- finding the 80th percentile spend:
 - `spendValues[Math.floor(spendValues.length \* 0.8)]`
- threshold fallback: `averageOrderValue \* 1.5`

- a customer is high-value if ``totalSpend >= highValueThreshold``

Business reasoning:

- top spenders drive disproportionate revenue
- they are strong candidates for loyalty and upsell

Recent Buyers

Calculated by:

- ``daysSinceLastOrder <= 30``

Business reasoning:

- warm audience
- most likely to respond quickly
- good for cross-sell or repeat purchase campaigns

At-Risk Customers

Calculated by:

- ``daysSinceLastOrder >= 60``

Business reasoning:

- these customers were previously active
- they are currently dormant
- highest potential for win-back campaigns

Weekend Shoppers

Calculated by:

- ``weekendOrders / orderCount >= 0.5``

Business reasoning:

- they have clear behavioral timing preferences

- good for Friday/Saturday promotions
- supports time-sensitive campaign activation

Why AI may select each segment

- Prompt mentions "weekend" → `weekend-shoppers`
- Prompt mentions "inactive" or "win back" → `at-risk`
- Prompt mentions "upsell", "premium", or "high value" → `high-value`
- Prompt mentions "recent purchase" → `recent-buyers`
- The AI has access to segment names, size, and revenue, and is asked to choose the best segment.

Data used in each segment

- `customerCount`
- `revenueContribution`
- `revenuePercent`
- `customerIds`

These values are used for:

- selection
- expected revenue projection
- audience size display
- snapshot creation

8. Revenue Prediction Logic

```
## `computeExpectedRevenue()`
```

File: StrategyEngine.ts

Actual formula

```
``ts
```

```
const avgOrderValue = intelligence.averageOrderValue || 45;

const segmentAOV = segment.customerCount > 0

  ? segment.revenueContribution / segment.customerCount

  : avgOrderValue;

const baseLiftMap: Record<SegmentId, number> = {

  'high-value': 0.05,

  'recent-buyers': 0.12,

  'at-risk': 0.25,

  'weekend-shoppers': 0.18,

};

const lift = baseLiftMap[segmentId] || 0.1;

const predictedConversions = Math.ceil(segment.customerCount * (lift * 0.8));

return Math.round(predictedConversions * segmentAOV * 1.15);

...
```

Explanation

- `segmentAOV`: average spend per customer in selected segment
- `lift`: segment-specific expected uplift
- `predictedConversions`: uses `lift \* 0.8` to make the conversion count conservative
- final revenue multiplies conversions by `segmentAOV`
- adds `15%` premium for campaign impact

Assumptions

- historical segment spend is a good proxy for future order value
- each segment has a fixed lift potential

- 80% of lift converts to actual conversions
- campaign impact is slightly above baseline with +15%

Limitations

- Does not use actual past conversion rates
- Ignores channel mix, message effectiveness, or cohort aging
- Uses static per-segment lift values hard-coded in StrategyEngine.ts
- No machine learning model or temporal forecasting
- Does not account for customer fatigue or campaign frequency

Frontend revenue formulas

There are additional ROI heuristics in:

- campaign-execution-center.tsx
- executive-dashboard.tsx

Examples:

- campaign cost computed as ``Math.max(Math.round(audienceCount * 0.08), 250)``
- projected revenue lift computed as ``expectedRevenue`` or a fallback estimate based on conversions and average spend
- after campaign revenue computed as ``baselineRevenue + projectedRevenueLift``
- ROI computed as ``Math.round(((projectedRevenueLift - campaignCost) / campaignCost) * 100)``

These are presentation-level projections, not backend truth.

9. Interview Questions and Answers

Product

1. What business problem does CampaignPilot solve?

- Ideal: It turns customer/order data into campaign strategy and ROI predictions without manual segment building.

- Follow-up: Which feature shows that value immediately?

- Testing: product understanding.

2. Who is the target user of this app?

- Ideal: retail marketers, e-commerce operators, founders, revenue ops.

- Follow-up: Why not a traditional CRM user?

- Testing: user personas.

3. Why is this product AI-native?

- Ideal: core flow relies on LLM reasoning in preview, goal, audience, and content generation.

- Follow-up: Which files prove it?

- Testing: architecture awareness.

4. How does it differ from a CRM?

- Ideal: CRM stores contacts and pipelines; this product invents campaign strategy and messaging.

- Follow-up: What does this product intentionally omit?

- Testing: product differentiation.

5. What is the core value proposition?

- Ideal: faster campaign ideation → higher confidence → revenue forecasting.

- Follow-up: Which UI component sells it?

- Testing: positioning.

Architecture

6. How is the app structured?

- Ideal: Next.js app router, api routes, components, lib, prisma.

- Follow-up: Why is lib separate from components?

- Testing: folder-level architecture.

7. What responsibility does StrategyEngine.ts have?

- Ideal: business logic, prompt analysis, revenue math, fallback content.
- Follow-up: Why not keep that in route handlers?
- Testing: separation of concerns.

8. What is the role of agents?

- Ideal: a simple AI agent pipeline for goal, audience, and content.
- Follow-up: Why use agents instead of direct route logic?
- Testing: modularity.

9. Why is 'OpenRouterProvider' used instead of the native OpenAI API?

- Ideal: it routes through OpenRouter and uses Gemini model.
- Follow-up: how is the base URL configured?
- Testing: integration knowledge.

10. Why is prisma.ts created?

- Ideal: share a single Prisma client instance and avoid extra connections.
- Follow-up: what is the global cache for?
- Testing: backend resource management.

AI

11. Describe the prompt used by preview route.

- Ideal: ask for best segment, reason why, campaign recommendation, and business impact in JSON.
- Follow-up: what fields are required?
- Testing: prompt engineering.

12. What happens if the AI returns text outside JSON?

- Ideal: the code searches response with regex and fails if no JSON found.
- Follow-up: how would you harden it?
- Testing: robustness.

13. Why do agents return JSON only?

- Ideal: parse reliability and direct data mapping.
- Follow-up: what is the parsing code line?
- Testing: implementation detail.

14. How is `GoalInterpreterAgent` different from `AudienceAgent`?

- Ideal: goal interprets intent/objective; audience chooses who to target.
- Follow-up: which one uses segment summaries?
- Testing: agent design.

15. What fallback exists if AI is unavailable?

- Ideal: template content from `getSegmentContent()` and default selection logic.
- Follow-up: where is that fallback defined?
- Testing: failure handling.

Backend

16. What does route.ts do?

- Ideal: creates snapshot, updates campaign, starts channel simulation.
- Follow-up: why create a snapshot?
- Testing: backend flow.

17. Why does `ChannelService.processCampaign()` not await send completion?

- Ideal: it returns accepted immediately and let async simulation continue.
- Follow-up: what are the risks?
- Testing: async architecture.

18. How are simulated events sent back to the app?

- Ideal: via route.ts.
- Follow-up: what metrics are updated?

- Testing: event flow.

19. How does polling work on the frontend?

- Ideal: page.tsx polls `/api/campaigns/status?id=...` every 1.5s.
- Follow-up: what condition stops polling?
- Testing: UI refresh.

20. Why is the campaign strategy stored as JSON string?

- Ideal: simple persistence of nested agent output.
- Follow-up: what are the drawbacks?
- Testing: database design.

Frontend

21. What does page.tsx orchestrate?

- Ideal: app state, steps, API calls, modals, preview and launch flow.
- Follow-up: name three major states it tracks.
- Testing: page-level complexity.

22. How is `AIExplainabilityPanel` used?

- Ideal: displays preview segment rationale and revenue opportunity.
- Follow-up: what does it render from `CampaignPreview`?
- Testing: UI contract.

23. Why is `DataSourcesModal` important?

- Ideal: entry point for real data ingestion
- Follow-up: what two data sources does it support?
- Testing: UX flow.

24. How is campaign status shown to the user?

- Ideal: `WorkflowProgress`, sidebar, and `CampaignExecutionCenter`.

- Follow-up: where is `campaignStatus` updated?
- Testing: state and UX.

25. How is ROI communicated visually?

- Ideal: `ExecutiveDashboard` charts and summary cards.
- Follow-up: which file calculates ROI?
- Testing: product storytelling.

Database

26. Why is `SegmentSnapshot.customerIds` a string?

- Ideal: snapshot persists arbitrary audience lists without a join table.
- Follow-up: how is it created?
- Testing: data modeling.

27. What is stored in `CampaignMetrics`?

- Ideal: sent, delivered, opened, clicked, conversions, revenue.
- Follow-up: which route updates these?
- Testing: analytics model.

28. What relation exists between `Campaign` and `DeliveryLog`?

- Ideal: one campaign has many delivery logs.
- Follow-up: why might that be useful?
- Testing: audit trail.

29. Why does `Order` have both `orderDate` and `createdAt`?

- Ideal: preserve reported purchase date and ingestion timestamp.
- Follow-up: how is `orderDate` parsed?
- Testing: data integrity.

30. How would you improve the snapshot model?

- Ideal: normalize to snapshot items or campaign audience table.
- Follow-up: what problem does it solve?
- Testing: schema evolution.

System Design

31. What are the scale limitations of this app?

- Ideal: in-process simulation, SQLite, polling, no queueing.
- Follow-up: which component is most fragile at scale?
- Testing: architecture scalability.

32. How would you make launch simulation production-ready?

- Ideal: move to background job worker, use real messaging channels, add retries.
- Follow-up: what tech would you choose?
- Testing: operational design.

33. What are the security gaps?

- Ideal: no auth, open callbacks, API routes exposed.
- Follow-up: how would you secure `callbacks/channel`?
- Testing: security awareness.

34. How would you improve AI prompt reliability?

- Ideal: use schema validation, JSON schema, safety checks, retry on parse fail.
- Follow-up: where would you implement it?
- Testing: prompt engineering maturity.

35. How would you handle multiple users?

- Ideal: add user accounts, multi-tenant data isolation, auth tokens.
- Follow-up: what would change in the database?
- Testing: multi-user design.

Scalability

36. Why is SQLite a limitation here?

- Ideal: not built for concurrent writes under load.
- Follow-up: what DB would you use instead?
- Testing: infrastructure choice.

37. What happens if two campaign launches occur simultaneously?

- Ideal: `ChannelService` may spawn simultaneous async loops, DB may be saturated.
- Follow-up: how to improve?
- Testing: concurrency.

38. How would you reduce frontend polling?

- Ideal: use WebSockets or server-sent events.
- Follow-up: what backend change is needed?
- Testing: real-time design.

39. How would you scale segment discovery?

- Ideal: move from querying all orders/customers to incremental aggregation or materialized views.
- Follow-up: what service can precompute segments?
- Testing: analytics scaling.

40. How do you make the AI pipeline fault-tolerant?

- Ideal: fallback content, retry loops, provider health checks.
- Follow-up: what if OpenRouter is down?
- Testing: resilience.

AI-specific

41. Why is `temperature` set to 0.7?

- Ideal: balance creativity with consistency.

- Follow-up: what would happen if it were 0.2?
- Testing: model tuning.

42. How would you ensure the model returns `selectedSegmentId`?

- Ideal: add explicit instructions and validation logic for the JSON field.
- Follow-up: what if it returns a different value?
- Testing: prompt guardrails.

43. Why does `AudienceAgent` sort segments by revenueContribution before fallback?

- Ideal: choose the highest-value default when AI cannot decide.
- Follow-up: is this always correct?
- Testing: fallback semantics.

44. What is the purpose of `buildAudienceReasoning()`?

- Ideal: produce human-readable rationale from quantitative segment data.
- Follow-up: where is it used?
- Testing: explainability.

45. How does `ContentAgent` use segment data?

- Ideal: it passes segmentName, criteria, expectedRevenue to prompt.
- Follow-up: what is the fallback if AI fails?
- Testing: dynamic content.

Technical debt

46. Where is the biggest technical debt?

- Ideal: campaign strategy stored as raw JSON string, weak callback auth, SQL-like snapshot string.
- Follow-up: how to refactor?
- Testing: debt recognition.

47. What is a fragile part of the code?

- Ideal: regex parsing of AI output.
- Follow-up: how to make it robust?
- Testing: reliability.

48. How would you test this product end-to-end?

- Ideal: ingest demo CSV, preview prompt, generate campaign, launch, verify metrics.
- Follow-up: which components would you mock?
- Testing: QA strategy.

49. How does the app enforce prompt intent?

- Ideal: `analyzePrompt()` uses keywords and length.
- Follow-up: why is this insufficient?
- Testing: NLP intent detection.

50. If you had 4 weeks, what would you build next?

- Ideal: auth, persistent customers/channels, real delivery provider, queue worker, multi-user analytics.
- Follow-up: what is the MVP priority?
- Testing: roadmap thinking.

10. Weaknesses

Current limitations

- AI parsing is brittle: uses regex to find `{...}` and `JSON.parse`
- No authentication or authorization
- Delivery is fully simulated, not real email/SMS
- Campaign audience snapshots are stored as JSON strings
- segment discovery is heuristic rather than adaptive
- UI state is large in `page.tsx`, making the page hard to maintain

- `SegmentSnapshot` relation to `Customer` appears conceptually incorrect
- Shopify integration is mocked in UI only
- no user accounts or multi-tenant separation

Technical debt

- `Campaign.strategy` is a string and should be normalized or typed
- route handlers contain business logic instead of delegating fully to services
- fallback content logic is duplicated between preview and content agent
- event callbacks are not authenticated or queued
- simulation runs in-process inside `ChannelService`; this is not production-safe

Areas that would fail at scale

- SQLite database under concurrent ingest/campaign load
- 1.5-second frontend polling across many clients
- in-process long-lived async `simulateDelivery()`
- `CustomerIntelligenceService.discoverSegments()` fetching all orders/customers every request
- no back-pressure for large CSV ingestion

Improvements for production

- Move to PostgreSQL or equivalent for concurrency
- Normalize campaign strategy/segment snapshot data
- Introduce background worker queue for campaign send and callback handling
- Replace polling with events or WebSockets
- Add proper auth, access control, and webhook signature verification
- Build real channel integrations instead of simulation
- Harden AI parsing with schema validation and retries
- Introduce a segment-audience table rather than storing arrays as strings
- Add server-side caching for segment summaries

11. AI-Native Development Story

How AI was used while building

- AI is not only an optional feature — it is the product's decision engine
- the app uses AI for:
 - prompt interpretation
 - audience selection
 - campaign content generation
 - campaign reasoning and explanations
- the code is organized around this pipeline:
 - agents
 - OpenRouterProvider.ts
 - route.ts

Why this qualifies as AI-native

- the user interacts with a blank prompt and gets an AI-crafted strategy
- the product depends on AI for core output, not just data fetching
- the value comes from model reasoning, not only from analytics
- there are multiple distinct AI touchpoints, not a single “chat” page

What decisions still required human judgment

- defining the core segments (`high-value`, `recent-buyers`, `at-risk`, `weekend-shoppers`)
- choosing revenue projection formulas
- designing campaign steps and UX flow
- selecting which data points to display
- deciding AI fallback templates and error behavior

In other words, humans designed the business rules, prompt scaffolding, and execution model, while AI delivers the strategy and messaging.

12. Executive Summary

CampaignPilot is an AI-native marketing dashboard built in Next.js that converts retail customer and order data into revenue-focused campaign strategy.

Key strengths:

- page.tsx orchestrates a guided workflow: ingest data, discover segments, analyze goals, generate strategy, launch, and display ROI
- CustomerIntelligenceService.ts discovers customer segments from historical order data
- OpenRouterProvider.ts connects to Gemini through OpenRouter for AI decisioning
- `GoalInterpreterAgent`, `AudienceAgent`, and `ContentAgent` form a modular AI pipeline
- campaigns are stored in `Campaign`, with metrics in `CampaignMetrics` and audience snapshots in `SegmentSnapshot`
- `ChannelService` simulates delivery while `callbacks/channel/route.ts` records event metrics

The product distinguishes itself from a CRM by focusing on campaign activation and revenue outcomes rather than contact management. It is AI-native because its core behavior is to reason through a marketing prompt, select the optimal audience, and generate campaign content, all through LLM-driven agents.

If asked in an interview, emphasize:

- you built a real pipeline from raw data to campaign launch
- you separated UI from AI/business logic
- you designed a lightweight agent architecture around `OpenRouterProvider`
- you created explainable outputs in `AIExplainabilityPanel`
- you balanced AI generation with fallback templates and deterministic revenue math

This is a full-stack product with clear user value: upload or connect data, tell the system your objective, and let AI produce a campaign recommendation plus a revenue story in under a minute.

